Implementing 2d Tori Algorithms on de Bruijn Graphs

Internal Report 4/94
May 1994

M. Nölle and G. Schreiber
Technische Universität Hamburg–Harburg
Technische Informatik I
Prof. Dr. Ing. H. Burkhardt
21071 Hamburg
Germany

e-mail: g-schreiber@tu-harburg.d400.de

In this paper we discuss embeddings of 2d tori in de Bruijn networks.

To our knowledge the best known embedding of 2d tori in N node de Bruijn graphs of *base two* has a dilation of $\log_2 N$ while the best theoretical lower bound is $O(\log\log N)$ ([1]). Heath claims in ([2]) that it is not possible to embed 2d tori in de Bruijn graphs with dilation less than $O(\log N)$, but up to now there doesn't seem to exist a proof.

After extending the scope to de Bruijn graphs of any base, we prove that a 2d torus is a spanning subgraph of the two dimensional de Bruijn graph. In conjunction with an embedding of one de Bruijn graph into another of different base this provides us with an embedding of 2d tori in fixed base B de Bruijn graphs with dilation $\log_B(\sqrt{N})$. This improves the so far known embeddings by a factor of $\frac{1}{2}$. We introduce a modification of the graph embedding measure *congestion*, the so-called *clocked congestion*. We demonstrate how to modify our embedding to reduce the clocked congestion. Finally, experimental results on a 64 processor system are given.

Abstract

In image processing the torus and the de Bruijn graphs are two popular structures. In this paper we discuss embeddings of 2d tori in de Bruijn networks. After extending the scope to de Bruijn graphs of any base, we prove that a 2d torus is a spanning subgraph of the two dimensional de Bruijn graph. In conjunction with an embedding of one de Bruijn graph into another of different base this provides us with an embedding of 2d tori in fixed base B de Bruijn graphs with dilation $\log_B(\sqrt{N}))$. To our knowledge, this improves the best known embedding by a factor of $\frac{1}{2}$. We introduce a modification of the graph embedding measure *congestion*, the so-called *clocked congestion*. We demonstrate how to modify our embedding to reduce the clocked congestion to one. Finally, experimental results on a 64 processor system are given.

Introduction

One of the major aims in the development of algorithms and their implementation is a high reusability of the code i.e, the code should be compilable and executable on (almost) any machine. However, in general it is quite difficult to satisfy this demand even on sequential machines. It is far more complicated in the case of parallel algorithms and machines. This is due to the fact that the properties of parallel machines are highly dependent on their interconnection structure, a problem sequential machines do not encounter. Thereby the parallelization or implementation of algorithms is often restricted to a certain class of networks. In order to overcome this hardware dependency, it is necessary to emulate one interconnection network by another.

In digital image processing many algorithms can efficiently be parallelized on either a torus or a de Bruijn network.

Data dependencies in image preprocessing algorithms are often *local* i.e, limited to a narrow area around each image element (pixel). Low- or highpass filters, filters for edge detection and certain invariant transformations are examples of those algorithms. They typically operate on square regions of 3×3 to 7×7 pixels. These algorithms are well suited for the 2d torus or other networks with nearest neighbour connection.

In contrary, higher level image processing algorithms, such as Fast Fourier, Fast Walsh Transformation or the Viterbi algorithm, have *global* data dependencies i.e, each output element depends on all input pixels. A network with low diameter is needed for an efficient parallelization.

The data flow graphs of the mentioned global algorithms are, upon others, isomorphic to an iterated de Bruijn graph. Thereby a parallel computer with de Bruijn interconnecting network can informally be said to be optimal for the parallelization of these algorithms.

Of course not all algorithms can be classified either local or global. There are examples of image processing algorithms needing local nearest-neighbour communication at one stage and having global data dependency at another. If we want to compute such an algorithm with time varying communication demands or both algorithms with local as well as global data dependencies on a parallel machine with fixed interconnecting network, we are faced with the problem which network to choose.

Choosing the 2d torus, one has to emulate a global network, in this case the de Bruijn graph. The embedding of de Bruijn graphs in torus networks suffers from the high diameter $O(\sqrt{N})$ of

2d tori. In a de Bruijn graph the maximal distance between any two vertices is $O(\log N)$. When embedding the de Bruijn graph into a torus this distance is increased to $O(\sqrt{N})$.

In the current paper we focus on the reverse strategy which is to emulate 2d tori on de Bruijn networks.

To our knowledge the best known embedding of 2d tori in de Bruijn graphs of *base two* has dilation of $\log_2 N$ ([3]) while the best theoretical lower bound is $O(\log \log N)$ ([1]). Some authors claim that it is not possible to embed 2d tori in de Bruijn graphs with dilation less than $O(\log N)$ but up to now there doesn't seem to exist a proof ([2]).

In [5] the scope of host graphs was extended to de Bruijn graphs of any base.

For de Bruijn graphs of *dimension* three or higher we could prove that 2d tori can't be embedded with dilation one ([6]).

A two dimensional de Bruijn graph can emulate any torus of the same size with dilation one. In conjunction with an embedding of de Bruijn graphs of base B^m into de Bruijn graphs of fixed base B, this provides us with an embedding of 2d tori in base B de Bruijn graphs with dilation $\log_B(\sqrt{N})$. Unfortunately the latter embedding is burdened with a congestion of B^{m-1} i.e, one edge of the base B de Bruijn graph has to emulate many edges of the base B^m de Bruijn graph. But this, as we will show, doesn't neccessarily involve such a high congestion for the overall embedding of 2d tori in de Bruijn graphs of base B. We will demonstrate how to reduce the overall congestion to one in a clocked communication model.

When constructing graph embeddings one is often faced with a trade-off between congestion and dilation. In order to achieve a 'good' result, it is necessary to judge the effect of congestion and dilation for the problem given. As described later, dilation can be taken as an equivalent to setup time and congestion corresponds to a decrease of available bandwidth. In general, without any knowledge of the particular problem, it is difficult to find a compromise between congestion and dilation.

However, in most parallel systems bandwidth is a crucial bottleneck. On the other hand an increase in setup time in many cases can be compensated by algorithmic means. When communication and calculation are properly scheduled so that both can be done in parallel, a longer setup time is preferable against a lower bandwidth. When reducing bandwidth the amont of calculations per communicated data item has to be increased in order to keep the proper schedule. For many algorithms this limits the degree of parallelism.

These conclusions lead us to a policy of optimizing the bandwidth rather than the setup time i.e, to reduce congestion rather than dilation. However, with the results presented in this paper, we are able to decrease congestion *without any loss in dilation*. This is done by means of a clocked model of communication. We briefly review our former results of embedding 2d tori in de Bruijn graphs with dilation $\log \sqrt{N}$ ([6],[5]). In this paper we focus on the congestion of these embeddings which can be reduced to one in a model of clocked communication.

In the following 1 we give a brief review of some basic definitions. A class of embeddings of 2d tori in de Bruijn graphs is presented in section 2. Their properties such as load, expansion and dilation are discussed. Section 3 introduces an alternative definition of congestion based on a clocked model of communication and presents two possible embeddings of the said class. First experimental results are given in section 4.

1 Basic Definitions

We describe a graph $G = (V, E)$ by a set V of vertices and a set $E \subseteq V \times V$ of edges. A graph is called *undirected* if for any edge the antiparallel edge is also element of E, otherwise G is called *directed*. Any directed graph can be easily transformed into an undirected one by adding the necessary edges.

Definition 1 (Torus) *The directed m dimensional* torus *graph $T(l_{m-1}, \ldots, l_0) = (V_M, E_M)$ with extension l_i in the ith dimension is given by*

$$V_M = \{(a_{m-1}, \ldots, a_0) | 0 \leq a_i < l_i \; \forall \, 0 \leq i < m\}$$

$$E_M = \{(a, b) | \exists i = 0 \ldots m-1 : b_i = (a_i + 1) \bmod l_i \wedge a_j = b_j \; \forall \, j \neq i\}.$$

The nodes of a de Bruijn graph are described by strings. The length n of the strings corresponds to the dimension of the de Bruijn graph. The range of the string elements (digits) is given by the base B of the de Bruijn graph. In the following strings are denoted by latin letters (e.g. u) whereas greek letters (e.g. $\alpha \in \{0, \ldots, (B-1)\} = \mathbb{Z}_B$) denote single digits. The concatenation of strings or digits respectively is written as the concatenation of the respective symbols (e.g. αu denotes the concatenation of the string u and the digit α). In order not to confuse string concatenation with multiplication, the latter shall be written with an centered dot.

A length n string u, $|u| = n$, of base B can be interpreted in different ways. Firstly, we can take u as the polyadic base B representation of an element of $\mathbb{Z}_{B^n} = 0, \ldots, (B^n - 1)$, i.e, $u = (\alpha_{n-1} \ldots \alpha_0) = \sum_{i=0}^{n-1} B^i \cdot \alpha_i$. Secondly, u can be seen as an element of the product set $(\mathbb{Z}_B)^n = \mathbb{Z}_B^n$. Please note that the definition of the plus operator differs in these two sets. The plus operator $(+)$ in $\mathbb{Z}_{B^n}$ acts modulo B^n. In $\mathbb{Z}_B^n$ plus $(\oplus)$ acts component by component modulo B.

Definition 2 (De Bruijn graph) *The directed base B* de Bruijn graph *$D_B(n) = (V_D, E_D)$ of dimension n is given by the node set $V_D = \mathbb{Z}_B^n$ and the edge set*

$$E_D = \{(\alpha u, u\beta) | u \in \mathbb{Z}_B^{n-1} \; \wedge \; \alpha, \beta \in \mathbb{Z}_B\}.$$

Any edge $(\alpha u, u\beta) \in E_D$ can be assigned an index $= \alpha u \beta$. This index is a one-to-one mapping of E_D to $\mathbb{Z}_B^{n+1}$.

In an undirected de Bruijn graph an edge is said to be in *shuffle direction* if this edge can also be found in the undirected de Bruijn graph. Otherwise the edge is in *unshuffle direction*.

In order to get an easy notation for paths in de Bruijn graphs we introduce the following operator τ.

Definition 3 (Take Operator τ) *The take operator $\tau_{k,i} s$, $s \in \mathbb{Z}_B^n$ takes k consecutive digits from s starting with the ith digit. All index operations are modulo n.*

$$\tau_{k,i}(s) = \tau_{k,i}(s_{n-1} \ldots s_0) = (s_{i+k-1} \ldots s_i)$$

For any two nodes $x, y \in \mathbb{Z}_B^n$ we denote by $s = xy = (x_{n-1} \ldots x_0 y_{n-1} \ldots y_0)$ a path from x to y. The path traverses the nodes $s_n, \ldots, s_0$ where $s_i = \tau_{n,i} s$, $i = n \ldots 0$. Please note that $(s_{i+1}, s_i) \in E_D$ by construction.

We call s the route from x to y. It is easy to see that the shortest route from 0 to $B^n - 1$ uses n intermediate nodes. That gives the worst case and establishes the logarithmic diameter of the de Bruijn graph.

There are several cases where we can find a shorter route for a given source and destination. If we consider two nodes $x = uv$, $y = vw1\mathbb{Z}_B^n$, where $|u| = k$, we get a route

$$s = uvw \tag{1}$$

of length $|u| = |w| = k$. It may be concluded from [4] for directed de Bruijn graphs that whenever a maximal suffix of the source address equals the prefix of the destination then (1) determines the shortest path.

Definition 4 (Embedding) *Let $G = (V_G, E_G)$ and $H = (V_H, E_H)$ be graphs. Let $\psi : V_G \to V_H$ be a mapping of vertices from the* guest graph *G to vertices of* host graph *H. ψ is called an* embedding *of G in H.*

An embedding can be characterized by two features called *load* and *expansion* ([3]). The load of an embedding gives the maximal number of vertices of the guest graph which are mapped to a single vertex of the host graph. In other words the load of an embedding tells us how many processors of the guest graph have maximally to be emulated by a single host processor. The expansion of an embedding gives the fraction the host graph nodes which are used by the guest graph. An expansion of one indicates that all vertices of H are used by G. If additionaly the load is one, the embedding is one-to-one.

Let $g' = (g_1, g_0) \in E_G$. Please observe that there might be more than one path between $\psi(g_1)$ and $\psi(g_0)$ in H even if we consider only shortest paths in H. In order to yield a well defined routing for a mapping ψ, we introduce a corresponding function $\psi' : E_G \to E_H \times E_H \times \ldots \times E_H$ which maps any edge of G to a single path in H (ψ' does not neccisscarily give a shortest path in H; even paths with loop are allowed).

Definition 5 (Dilation) *Let (ψ, ψ') be an embedding of G in H. The* dilation *of the embedding is the maximal length of all paths in $\psi'(E_G)$:*

$$\text{dilation}(\psi') = \max_{g' \in E_G} \left(\left| \psi'(g') \right| \right).$$

In an embedding with dilation one, neighbours in G are mapped to neighbours in H. If in addition the load is one, G is a subgraph[1] of H. The effect of an increase in dilation is not necessarily a decrease of bandwidth as one might suspect. By thinking of the processors along a path in H as a pipeline, it is obvious that a higher dilation (a longer pipeline) results in a longer setup time rather than a lesser throughput (a smaller bandwidth). The effect on bandwidth is described by congestion.

[1] If we do not consider parallel edges and loops in G.

Definition 6 (Simultaneous Congestion) *Let (ψ, ψ') be an embedding of G in H. The* simultaneous congestion *of the embedding is the maximal number of edges of G which are routed through a single edge of H:*

$$\text{congestion}(\psi') = \max_{h' \in E_H} \left(\left| \{ g' \in E_G \mid h' \in \psi'(g') \} \right| \right).$$

The congestion of an embedding tells us how many edges of G are mapped to paths that have to share a single edge in H. In the worst case this implies a decrease in effective bandwidth.

2 Embedding 2d tori in de Bruijn graphs

In this section we start with a class of embeddings of $M \times M$ tori in de Bruijn graphs of base M. Successively, the de Bruijn graph of base M can be embedded in a de Bruijn graph of fixed base and same size. The first embedding introduces two permutations which can be chosen arbitrarily within certain constraints. In section 3 we demonstrate the effect these permutations have on the overall embedding.

Theorem 1 *Let $T(M, M)$ be the $M \times M$ undirected torus graph and $D_M(2)$ the undirected base M de Bruijn graph of dimension 2, M even. Let further be π_0 and π_1 any permutations of $\mathbb{Z}_M$ with the property that for all $(a, b) \in V_T$: $\pi_0(a) = \pi_1(b) \Longrightarrow a \bmod 2 = b \bmod 2$. (2) The embedding $\psi : T(M, M) \to D_M(2)$ given by*

$$\psi(a,b) \quad = \quad \begin{cases} \pi_0(a)\pi_1(b) & : \quad (a+b) \bmod 2 = 0 \\ \pi_1(b)\pi_0(a) & : \quad (a+b) \bmod 2 = 1 \end{cases}$$

has load *1,* expansion *1 and* dilation *1.*

Please recall that $\pi_0(a)\pi_1(b)$ denotes the concatination (*not* the multiplication) of the strings returned by $\pi_0(a)$ and $\pi_1(b)$.

Proof In order to prove load and expansion one we will show that ψ is a one-to-one mapping. Since both graphs have the same number of nodes, we have to show that $\psi(a, b) = \psi(a', b')$ implies $(a, b) = (a', b')$.

In the case of $(a + b) \bmod 2 = (a' + b') \bmod 2$ we get

$$\begin{aligned} \psi(a,b) = \psi(a',b') \quad &\Longrightarrow \quad \pi_0(a) = \pi_0(a') \quad \wedge \quad \pi_1(b) = \pi_1(b') \\ &\Longrightarrow \quad a = a' \quad \wedge \quad b = b' \\ &\Longrightarrow \quad (a,b) = (a',b'). \end{aligned}$$

In the case of $(a + b) \bmod 2 \neq (a' + b') \bmod 2$

$$\begin{aligned} \psi(a,b) = \psi(a',b') \quad &\Longrightarrow \quad \pi_0(a) = \pi_1(b') \quad \wedge \quad \pi_1(b) = \pi_0(a') \\ &\overset{(2)}{\Longrightarrow} \quad a \bmod 2 = b' \bmod 2 \quad \wedge \quad b \bmod 2 = a' \bmod 2 \\ &\Longrightarrow \quad (a+b) \bmod 2 = (a'+b') \bmod 2 \end{aligned}$$

we get a contradiction. So ψ is one-to-one.

For the analysis of dilation let (a,b) and (a',b') be neighbours in $T(M,M)$. The following definition of the according routing function ψ' maps the edges of $T(M,M)$ to edges of $D_M(2)$. The edges of $D_M(2)$ are give by their indices.

$$\psi'((a,b),(a',b')) = \begin{cases} \pi_0(a)\pi_1(b)\pi_0(a') & : & (a+b) \bmod 2 = 0 & b = b' & \text{shuffle} \\ \pi_1(b')\pi_0(a)\pi_1(b) & : & (a+b) \bmod 2 = 0 & a = a' & \text{unshuffle} \\ \pi_1(b)\pi_0(a)\pi_0(b') & : & (a+b) \bmod 2 = 1 & a = a' & \text{shuffle} \\ \pi_0(a')\pi_1(b)\pi_0(a) & : & (a+b) \bmod 2 = 1 & b = b' & \text{unshuffle} \end{cases} \quad (3)$$

The last column indicates the direction of the according edge in $D_M(2)$.

Obviously, $\psi'((a,b),(a',b'))$ is the index on an edge from $\psi(a,b)$ to $\psi(a',b')$ in $D_M(2)$. Thus the dilation of the embedding is one.

■

Theorem 1 establishes the fact that $T(M,M)$ is a spanning subgraph of $D_M(2)$. However, the de Bruijn graph $D_M(2)$ is of small practical value because of it has a high non-constant node degree of $2 \cdot M$. To take advantage of the presented embedding, we have to provide an additional embedding of $D_M(2)$ in a bounded degree de Bruijn graph $D_B(2 \cdot m)$, where $M = B^m$. $D_M(2)$ and $D_B(2m)$ have the same number of nodes:[2] $|V_{D_M(2)}| = M^2 = (B^m)^2 = |V_{D_B(2 \cdot m)}| = B^{2 \cdot m}$.

The embedding is done by expanding the base B^m representation of every digit of a node of the $D_M(2)$ into a base B representation. Therby we get the string of an node in $D_B(2 \cdot m)$. With this any edge $(\alpha\delta, \delta\beta) \in E_{D_M(2)}$ is mapped to a route $s = \alpha\delta\beta$ (by using (1)) of length m which equals the index of that edge. Thus the dilation of this embedding is m while obviously load end extension are one. If we use those two presented embeddings one after another, we gain an embedding of $T(M,M)$ in $D_B(2 \cdot m)$ with load and expansion one while the dilation is $m = \log_B(\sqrt{|V_T|})$.

3 Congestion of the embedding

The congestion of an embedding provides us with an upper bound for the number of edges of G being mapped to a single edge in H. Taking H as a processor network emulating G, one might suppose that the effective bandwidth in H is congestion(ψ') times lower than the physical bandwidth. Indeed, if we assume that the communication uses all edges on all routes *at the same time*, this conjection is correct. In a more realistic model where the edges of a path are used one after another and in case of all messages taking about the same time to be transmitted via an edge, we can define a more realistic upper bound for the number of messages passed through a single edge at a time.

As a simple example we take a ring of three vertices embedded in another ring of three as shown in Figure 1. Clearly this embedding has load and expansion one while dilation and congestion is two.

We now introduce a model of communication where the time required to send a message from one processor to one of its direct neighbours is identical for all messages. In this model we gain

[2] To be precise, their set of nodes are identical with respect to the polyadic interpretation of the node sets.

a clocked communication in the sense that in the first step of communication all first edges of all paths are used, in the next step all second edges are used and so on. Naturally, conflicts between messages using the same edge (ie. congestion) can occur only when these messages use that edge at the same point in time. In our example shown in Figure 1 the clocked congestion is one while the simultaneous congestion is two.

Definition 7 (Clocked Congestion) *Let (ψ, ψ') be an embedding of G in H. The* clocked congestion *of the embedding is the maximal number of edges of G being routed through the same edge of H at a time.*

$$\text{congestion}_c(\psi') = \max_i \left(\max_{h' \in E_H} \left(\left| \{ g' \in E_G \mid s = \psi'(g'), (s_{i+1}, s_i) = h' \} \right| \right) \right)$$

How realistic is the model of clocked communication? The assumption that all messages need about the same time for one hop[3] is justified under the following circumstances:

- All vertices (processors) and all edges (communication lines) have the same physical properties, i.e, bandwidth and setup time.
- All messages are of the same size.
- For each communication step all edges are used by the same number of paths. Otherwise the effective bandwidth per message wouldn't be identical for all messages and their transmitting time would deviate.

These constraints can be disposed off by synchronizing all processors ahead of each communication step. The synchronization does not have to be global; it is sufficient to perform a local synchronization between neighbours. This can be done easily by waiting for all incoming messages before starting to send the messages of the next step. The local synchronization ensures that all messages keep the order they would have had in case of a global synchronization. Thus local and global synchronization have the same effect towards congestion. Of course, we prefer to perform a local rather than a global synchronization, because the global synchronization is much more expensive in terms of processing time[4].

The method of local synchronization can be used efficiently even if the three constraints above are not strictly but almost fulfilled. In this case the time required for the synchronization can be traded off against the advantage of a higher effective bandwidth due to reducing the congestion. However, the less the constraints are fulfilled the longer one has to wait for the last incoming message to arrive and efficiency decreases. In case of the constraints being far away from reality, it might be better to return to the simultaneous congestion model.

In section 2 we presented an embedding of $T(M, M)$ in $D_B(2m)$ with dilation one. Now we will investigate the (clocked) congestion of this embedding. This is highly dependent on the functions π_0 and π_1 we choose. Please note that the constraint (2) is fulfilled for arbitrary permutations whenever we have $\pi_0 = \pi_1$.

In the following we dicuss two different examples of those permutations. The simpliest choice for the permutations π_0, π_1 one can think off is given by the identity permutation (id).

[3] A hop is a communication between to neighbour processors.

[4] A global synchronization is at least of order O(d(G)) with d(G) being the diameter of the graph G.

Theorem 2 *The embedding of section 2 with $\pi_0 = \pi_1 = id$ has a load, expansion of one and clocked congestion of two while the dilation is m.*

The related proof is of very technical nature. Therefore it is given in the appendix. We concentrate on a permutation that reduces the clocked congestion to one for de Bruijn graphs of base three and higher.

We introduce a function ϕ which will be employed to serve as π_0 and π_1.

Definition 8 *Let $a = (\alpha_{n-1} \dots \alpha_0) \in \mathbb{Z}_{B^n}$. The function $\phi_n : \mathbb{Z}_{B^n} \to \mathbb{Z}_{B^n}$ with $n > 0$ and $B > 1$ is given by*

$$\phi_n(a) = \begin{cases} \beta\, \phi_{n-1}(\alpha_{n-2} \dots \alpha_0) & for \quad n > 1 \\ a & for \quad n = 1. \end{cases}$$

with $\beta = (a - \alpha_{n-1}) \bmod B$ being the first (most significant) digit of the polyadic base B representation of $\phi_n(a)$.

With the function ϕ we can now state our main Theorem:

Theorem 3 *The embedding presented in section 2 with $\pi_0 = \pi_1 = \phi_m$ embeds $T(M, M)$ in $D_B(2m)$, $B > 2$, $M = B^m$, with load, expansion and clocked congestion of one while the dilation is m.*

In order to give a staright forward proof of Theorem 3 we need two Auxiliaries.

Auxiliary 4 *ϕ_n is one-to-one.*

The proof of this Auxiliary can be found in the appendix.

Auxiliary 5 *For all $0 \le i < n$, $a, b \in \mathbb{Z}_{B^n}$, $a \pm 1 = a'$, $b \pm 1 = b'$, $B > 2$:*

$$(aa') \neq (bb') \quad \Longrightarrow \quad \tau_{n+1,i}\left(\phi_n(a)\, \phi_n(a')\right) \neq \tau_{n+1,i}\left(\phi_n(b)\, \phi_n(b')\right).$$

Proof (By induction)

For $n = 1$ Auxiliary 5 holds because of $(aa') \neq (bb')$.

Let Auxiliary 5 hold for n. (4)

Assume Theorem 5 does not hold for $n + 1$. So there exists an $i = 0 \dots n$ and $a, b, a', b' \in \mathbb{Z}_{B^{n+1}}$ with $aa' \neq bb'$ that (5)

$$\tau_{n+2,i}(\phi_{n+1}(a)\, \phi_{n+1}(a')) = \tau_{n+2,i}(\phi_{n+1}(b)\, \phi_{n+1}(b')). \tag{6}$$

With some digits $\delta, \delta', \rho, \rho' \in \mathbb{Z}_B$ (6) can be written as

$$\begin{aligned} \tau_{n+2,i}(\phi_{n+1}(a)\, \phi_{n+1}(a')) = \quad & \tau_{n+2,i}(\phi_{n+1}(b)\, \phi_{n+1}(b')) \\ \Longrightarrow \quad \tau_{n+2,i}(\delta\, \phi_n(a)\, \delta'\, \phi_n(a')) = \quad & \tau_{n+2,i}(\rho\, \phi_n(b)\, \rho'\, \phi_n(b')). \end{aligned} \tag{7}$$

For $i = n$ we can separate (7) into $\phi_{n+1}(a) = \phi_{n+1}(b)$, $\delta = \rho$ and $\delta' = \rho'$. With Theorem 4 we get $\phi_{n+1}(a) = \phi_{n+1}(b) \Longrightarrow a = b$ and with equation (5): $a = b \Longrightarrow a' \neq b'$.
Without loss of generality let $a' = a + 1$ and $b' = a - 1$. This leads to

$$\begin{aligned} \delta' = (a' - \alpha'_{n-1}) \bmod B &= (b' - \beta'_{n-1}) \bmod B = \rho' \\ (a + 1 - \alpha'_{n-1}) \bmod B &= (a - 1 - \beta'_{n-1}) \bmod B \\ (\alpha'_{n-1} - \beta'_{n-1}) \bmod B &= 2. \end{aligned} \tag{9}$$

α'_{n-1} is the most significant digit of $(a + 1)$. This digit can't differ by two from β'_{n-1}, the most significant digit of $(a - 1)$. So (9) is false for $B > 2$.[5] Since the assumption of $i = n$ leads to a false statement, we can restrict i to $n - 1 \ldots 0$. This enables us to simplify (7) a little by disposing of δ and ρ:

$$\tau_{n+2,i}(\phi_n(a)\,\delta'\,\phi_n(a')) = \tau_{n+2,i}(\phi_n(b)\,\rho'\,\phi_n(b'))$$

By separating we get $\tau_{n+1,i}(\phi_n(a)\,\phi_n(a')) = \tau_{n+1,i}(\phi_n(b)\,\phi_n(b'))$, with is a contradiction to (4). There is no i to fulfill (6) and thus Theorem 5 holds for $n + 1$.

■

We can continue with the proof of Theorem 3.

Proof The load, expansion and dilation is covered by section 2.

To discuss congestion, we can restrict on all paths of one direction, e.g. the shuffle direction, because in a model of clocked bidirectional communication each edge can be used both ways without interference in each step.

A path s from (a, b) to a 2d torus neighbour (a', b') in shuffle direction has either

$$(a + b) \bmod 2 = 0 \wedge b = b' \quad \text{or} \quad (a + b) \bmod 2 = 1 \wedge a = a'.$$

In any case the route is of the type $s = \phi_n(a)\,\phi_n(b)\,\phi_n(a')$ with $a' = a \pm 1$.
The indices of the edges used by s are $= \tau_{2n+1,i}(\phi_n(a)\,\phi_n(b)\,\phi_n(a'))$, $i = n - 1 \ldots 0$
Assume there exists c, d, c' with $c' = c \pm 1$, $(aba') \neq (cdc')$ such that for at least one step $i \in \mathbb{Z}_n$ both paths use the same edge $\tau_{2n+1,i}(\phi_n(a)\,\phi_n(b)\,\phi_n(a')) = \tau_{2n+1,i}(\phi_n(c)\,\phi_n(d)\,\phi_n(c'))$.

By separating this equation get $\phi_n(b) = \phi_n(d) \overset{4}{\Longrightarrow} b = d \Longrightarrow (a, a') \neq (c, c')$ and

$$\tau_{n+1,i}(\phi_n(a)\,\phi_n(a')) = \tau_{n+1,i}(\phi_n(c)\,\phi_n(c')).$$

The last equation is a contradiction to Theorem 5. Thus no edge is used by more than one path at a time and the clocked congestion is 1.

■

Due to the limitations of cut sheet paper dimensions it is not possible to give an example of an embedding according to Theorem 3. The smallest non-trivial embedding handles $3^4 = 81$ vertices.

[5] In case of $B = 2$ Equation (9) is no contradiction because α'_{n-1} and β'_{n-1} might very well be equal.

4 Experimental Results

The presented emdeddings have been implemented on a 64 processor (T805, 30MHz) system with $D_2(6)$ interconnection network. Due to the base of two both embeddings yield identical efficiencies.

In the final version of this paper we will give detailed experimental results on the parallelization of the Channon algorithm [3] for matrix multiplikation.

For the time being we can give preliminary data on the parallelization of an image processing algorithm for the calculation of invarinat features [5]. This algorithm is used for object recognition and has a data dependency as indicated in Figure 2. The parameter k gives the distance of the data elements requiered for the computation of the center element. Figure 3 gives the resulting efficiency for an 512×512 frame being processed on 64 processors with varying parameter k. The curve marked *2d-torus* belongs to the embedding of $T(8,8)$ in $D_2(6)$ with $\pi_0 = \pi_1 = id$. The second curve marked *de Bruijn* gives the results of a naive reference embedding where the bits of the 2d torus coordinates have been interleaved to form the index of a node in the de Bruijn graph.

Each processor holds a 64×64 subframe. It can be clearly seen that for values of $k \leq 64$ (i.e, next neighbour communication in the 2d torus) the embedding yields a significant rise in efficiency (Base for the calculation of the efficiency is the sequential runtime on a single processor). We expect our results with the channon algorithm to give even better efficiencies because of its directed 2d torus communication pattern.

5 Conclusion

In this paper we have analysed a quite general class of 2d torus graph embeddings into de Bruijn graphs. Although the embedding is burdened with dilation $\log_B \sqrt{N}$ our implementational results of real live image processing applications show the efficiency of the presented methods.

With the clocked communication model we may consider the clocked congestion which introduces a more realistic measure in some applications. The low constant clocked congestion of the two aformentioned embeddings emphasize its usefulness for at least moderate sized parallel systems. This becomes even more important if algorithms with varying communication demands are considered. In this case we have to find a compromise in the system and the algorithmic layout in order to optimize the overall performance.

Until now for us it was not possible to gain full efficiency from the congestion one embedding. Since this embedding only achieves congestion one in de Bruijn graphs of base three or higher we can use that result only partially on nowadays Transputer systems. In future we hope to find an embedding into de Bruijn graphs of base two with clocked congestion one by choosing π_0 and π_1 different.

figures

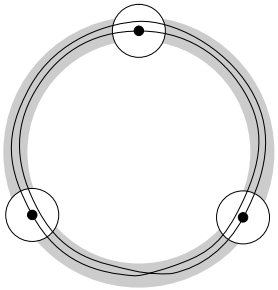


Figure 1: Embedding a ring with three vertices in another with load 1, expansion 1, dilation 2, congestion 2 and clocked congestion 1.

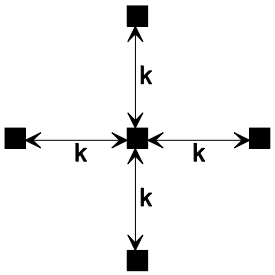


Figure 2: Data dependency with parameter k.

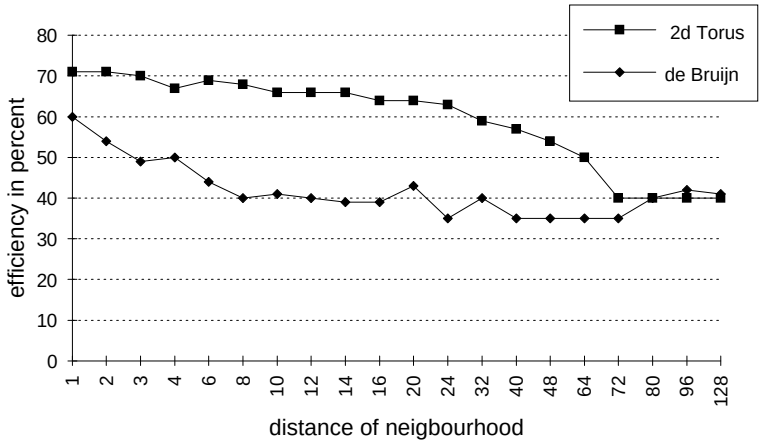


Figure 3: Efficiency of 2d torus and naive embedding.

Appendix

We now give the proof of Theorem 2. In the sequel we use the abreviation $\tau_{k,i}(s) = s_{k,i}$.

Let $x = (x_{m-1} \ldots x_0) \in \mathbb{Z}_{B^m}$, then $x' = x + 1$ may be written in a polyadic base B notation as

$$x' = \begin{cases} (0 \ldots 0) & : \quad x = B^m - 1 \\ (x_{m-1} \ldots x_{r+1}(x_r + 1)0 \ldots 0) & : x_r < B - 1 \wedge (x_{r-1} \ldots x_0) = B^r - 1. \end{cases} \tag{10}$$

Equally we get $x' = x - 1$

$$x' = \begin{cases} ((B-1) \ldots (B-1)) & : \quad x = 0 \\ (x_{m-1} \ldots x_{r+1}(x_r - 1)(B-1) \ldots (B-1)) & : x_r > 0 \wedge (x_{r-1} \ldots x_0) = 0. \end{cases} \tag{11}$$

Proof Theorem 2 (clocked congestion)
With $\pi_0 = \pi_1 = id$ the mapping ψ results in

$$\psi(x, y) \quad = \quad \begin{cases} xy : x + y \text{ even} \\ yx : \quad \text{else} \end{cases} \quad .$$

Without loss of generality we assume $(x + y) \bmod 2 = 0$. Since all paths $s = xyx'$ from Eq. (3) use the shuffle direction and do not interfere with paths in unshuffle direcetion ($s = x'yx$) from Eq. (3)) we may calculate the clocked congestion for the former onces.

We first show that torus edges starting from nodes with different remainders modulo 2 are mapped to non conflicting paths (in the clocked model).

Let $a, b, x, y \in \mathbb{Z}_{B^m}$ with

$$x + y \bmod 2 = 0 \iff x \bmod 2 = y \bmod 2 \implies \psi(x, y) = (xy),$$

$$a + b \bmod 2 = 1 \iff a \bmod 2 \neq b \bmod 2 \implies \psi(a, b) = (ba).$$

Let $x' = x \pm 1 \bmod M$, $a' = a \pm 1 \bmod M$ and $y' = y \pm 1 \bmod M$, $b' = b \pm 1 \bmod M$. If the paths $s = xyx'$ and $s' = bab'$ share an edge $s_{2 \cdot m+1,k} = s'_{2 \cdot m+1,k}$ for $k \in \{m-1, \ldots, 0\}$, we get $y = a$ and therefore $y \bmod 2 = a \bmod 2$ holds. Since we assumed $a \bmod 2 \neq b \bmod 2$ and $x \bmod 2 = y \bmod 2$ it follows that $x \bmod 2 \neq b \bmod 2$ and $x_0 \neq b_0$. From here we conclude that all edges forming the paths s and s' are different for all $k \in \{m-1, \ldots, 0\}$. For the case $s = y'xy$ and $s' = a'ba$ the same follows from $y'_0 \neq a'_0$ analogeously.

If two paths $s = xyx'$ and t of the embedding share an edge in the de Bruijn graph $D_B(2 \cdot m)$ there exists at least one $k \in \{m-1, \ldots, 0\}$ so that $s_{2 \cdot m+1,k} = t_{2 \cdot m+1,k}$. This implies $t_{m,m} = y$.

We may set $t = zyz'$, $z \in \mathbb{Z}_{B^m}$ and $z' = z \pm 1 \bmod M$. With $u = xx'$ and $v = zz'$ we get

$$\begin{array}{rrcl} & s_{2 \cdot m+1,k} & = & t_{2 \cdot m+1,k} \\ \iff & u_{m+1,k} & = & v_{m+1,k} \\ \iff & x_k \ldots x_0 {x'}_{m-1} \ldots {x'}_k & = & z_k \ldots z_0 {z'}_{m-1} \ldots {z'}_k \\ \implies & x_0 & = & z_0 \end{array} \tag{12}$$

Case $x \bmod 2 = 0$: Since $x_0 \bmod 2 = 0$ and M, B are even we have $x_0 + 1 < B$.

Case $x' = x + 1$: With $x'_0 = x_0 + 1$ follows $x' = (x_{m-1} \ldots x_1(x_0 + 1))$. (14)

Case $k > 0$: From (12) and (14) it follows that

$$x'_k = x_k = z_k = z'_k \quad \text{and} \tag{15}$$

$$x = (z'_{m-1} \ldots z'_k z_{k-1} \ldots z_0). \tag{16}$$

$z' = z + 1$ results in $x = z$. Therefore let $z' = z - 1$. From Eq. (11) we get two possible setting for z'.

Case $z = 0$: $\Longrightarrow z_k = 0 \neq z'_k = B - 1$. In conjunction with Eq. (15) this gives a contradiction.

Case $z \neq 0$: This implies $z = (z_{m-1} \ldots z_r 0 \ldots 0)$ for $z_r > 0$ and we get

$$z' = z - 1 = (z_{m-1} \ldots z_{r+1}(z_r - 1)\,(B-1) \ldots (B-1)).$$

Case $k \leq r$: $\stackrel{(16)}{\Longrightarrow} x = (z_{m-1} \ldots z_{r+1}(z_r - 1) z'_{r-1} \ldots z'_k 0 \ldots 0)$. For $k \neq r$ follows $z'_k = (B-1) \neq z_k = 0$. For $k = r$ we get $z'_k = z_k - 1 \neq z_k$. This is a contradiction to (15).

Case $k > r$: $\stackrel{(16)}{\Longrightarrow} x = (z_{m-1} \ldots z_k z_{k-1} \ldots z_0) \Longrightarrow x = z$.

In total for the case $x \bmod 2 = 0$ and $x' = x + 1$ either the first $(m - r - 1)$ edges of the paths $xy(x+1)$ and $xy(x-1)$ are congested or there is no congestion.

Case $k = 0$: Using Eq. (12) leads to $x' = z'$. $z' = z + 1$ and invoking Eq. (12) again gives $x = z$. Therefore take $z' = z - 1$. From here we conclude

$$\left.\begin{array}{rcl} x' & = & (x+1) \bmod M \\ z' & = & (z-1) \bmod M \\ x' & = & z' \end{array}\right\} \Longrightarrow (x+2) \bmod M = z$$

$$\Longrightarrow (x_0 + 2) \bmod B = z_0$$

$$\stackrel{(12)}{\Longrightarrow} B = 2.$$

If $B > 2$ the last edge of every path is used only once. In the case of $B = 2$ the nodes x and $z = x + 2$ are using this edge to reach $x' = x + 1$ and $z' = z - 1$.

Case $x' = x - 1$: For $k > r$ it has already been proven that the edges to $x - 1$ and $x + 1$ are congested. Following the same argumentation as befor the case $r \geq k \geq 0$ gives a contradiction if $B > 2$. $B = 2$ and $k = 0$ result in a common last edge of the path from $z = x - 2$ to $z' = z + 1$.

Case $x \bmod 2 = 1$: The proof is with $x' = x - 1$ analogeous to the case $x \bmod 2 = 0$.

Summerizing all cases we conclude that for the given embedding any edge from the de Bruijn graph $D_B(2 \cdot m)$ is used by at most two paths at the same time.

■

Proof Theorem 4 (By Induction)

Since ϕ_1 is the identity, Theorem 4 holds for $n = 1$.

Let Theorem 4 hold for n. (18)

To perform the induction step we have to show $\phi_{n+1}(a) = \phi_{n+1}(b) \Rightarrow a = b$.

$$\phi_{n+1}(a) = \phi_{n+1}(b) \implies \delta = \rho \quad \wedge \quad \phi_n(a) = \phi_n(b) \tag{19}$$

$$\begin{aligned} \phi_n(a) = \phi_n(b) &\implies \phi_n(\alpha_{n-1} \ldots \alpha_0) = \phi_n(\beta_{n-1} \ldots \beta_0) \\ &\overset{(18)}{\implies} \alpha_i = \beta_i \quad i = 0 \ldots n-1 \end{aligned} \tag{20}$$

$$\begin{aligned} \delta = \rho &\implies (a - \alpha_n) \bmod B = (b - \beta_n) \bmod B \\ &\implies (a - b) \bmod B = (\alpha_n - \beta_n) \bmod B \\ &\overset{(20)}{\implies} 0 = (\alpha_n - \beta_n) \bmod B \\ &\overset{(20)}{\implies} a = b \end{aligned}$$

■

References

[1] S.N. Bhatt, F.R.K. Chung, J-W. Hong, F.T. Leighton, and A.L. Rosenberg. Optimal simulations by butterfly networks. *Proc. 20'th ACM Symp. on Theory of Computing*, pages 192–204, 1988.

[2] L.S. Heath. Lower bounds for graph embeddings via algebraic topology. *5th Annual ACM Symposium on Parallel Algorithms and Archetectures*, pages 311–317, June 1993.

[3] F.T. Leighton. *Introduction to Parallel Algorithms and Architectures.* Morgan Kaufman Publishers, San Matio, California, 1992.

[4] Z. Liu. Optimal routing in the de bruijn networks. In *IEEE 10th Int. Conf. on Distributed Computing Systems*, pages 537–544, Paris, 1990. IEEE.

[5] H. Schulz-Mirbach M. Nölle, G. Schreiber. Efficient parallel algorithms for the extraction of image features with adjustable invariance properties. *Workschop on Orders, Algorithms and Applications (ORDAL'94), Lyon, France, (submitted)*, July 1994.

[6] G. Schreiber. Algorithms for self initialisation of communication networks and embeddings of subgraphs in logarithmic networks., Februar 1993. Diplom thesis, Technische Informatik I, Technische Universität Hamburg-Harburg, (written in german).